

AI Mock Interview

An AI-powered mock interview platform that helps students and job seekers practice for job interviews using a **Multi-AI evaluation pipeline** (Google Gemini + OpenAI + Cohere). Upload your resume, select interview rounds, and experience a realistic, adaptive interview with personalized questions and detailed performance feedback.

✦ Features

Core Features

- **Session-Based Access** — Lightweight session management with unique 8-character hex IDs (no password required)
- **Interview Spaces** — Create dedicated spaces for different companies/positions with uploaded resumes
- **AI Resume Analysis** — Automatically extracts text from PDF/DOCX resumes and generates an AI-powered summary relevant to the target job
- **AI-Generated Questions** — Personalized, context-aware interview questions based on your resume, job description, experience level, and round type
- **Adaptive Questioning** — Questions dynamically adjust based on your previous answers (probing deeper on weak responses, progressing to harder topics on strong ones)
- **3-Phase Interview Flow** — Warm-up → Core → Closing phases for a realistic interview experience
- **Follow-up Questions** — Smart follow-up questions that probe deeper into your answers

Multi-AI Evaluation Pipeline (3-AI Consensus)

- **Step 1 — Gemini (Evaluator A)**: Acts as a Senior Technical Lead, providing evidence-based evaluation with specific answer quotes
- **Step 2 — OpenAI (Evaluator B)**: Acts as a Hiring Manager, providing an independent parallel evaluation
- **Step 3 — Cohere (Synthesizer)**: Acts as a Chief Talent Officer, synthesizing both evaluations into a single definitive report with a consensus verdict
- **Graceful Fallback**: Works with Gemini alone if OpenAI/Cohere keys are not configured

Interview Rounds

- **HR Round** — Behavioral questions, motivation, cultural fit
- **Technical Round** — Data structures, algorithms, system design, coding concepts
- **Final Round** — Comprehensive assessment combining HR + Technical + Cultural fit
- **Custom Rounds** — Support for any additional round types

Experience-Level Calibration

- **Fresher** — 10 questions, foundational difficulty, focus on fundamentals and growth potential
- **Intermediate** — 12 questions, moderate difficulty, focus on practical experience and design patterns
- **Experienced** — 15 questions, high difficulty, focus on architecture, leadership, and trade-offs

Additional Features

- **Resume Upload** — Supports PDF and DOCX formats (max 10MB)
- **Resume Download** — Download previously uploaded resumes with path traversal protection
- **Performance Dashboard** — Track your performance across all interview spaces
- **User Profile** — Manage your name and session details
- **Content Sanitization** — All user-generated content sanitized with DOMPurify + JSDOM for XSS protection
- **Markdown Rendering** — Interview summaries and job descriptions rendered as formatted HTML via `marked`
- **Email Integration** — Gmail-based email support via Nodemailer

 Tech Stack

Category	Technology
Runtime	Node.js (v18+)
Framework	Express.js
Templating	EJS (Embedded JavaScript)
Database	MongoDB with Mongoose ODM
AI — Primary	Google Gemini AI (<code>@google/genai</code> , model: <code>gemini-2.5-flash</code>)
AI — Secondary	OpenAI (<code>openai</code> , model: <code>gpt-4o-mini</code>) — <i>optional</i>
AI — Synthesis	Cohere (<code>cohere-ai</code> , model: <code>command-a-03-2025</code>) — <i>optional</i>
File Uploads	Multer (PDF & DOCX, 10MB limit)
PDF Parsing	<code>pdf-parse</code>
DOCX Parsing	<code>mammoth</code>
Content Sanitization	DOMPurify + JSDOM
Markdown	<code>marked</code>
Session Management	<code>cookie-session</code> (30-day expiry)
Security	<code>express-rate-limit</code> , <code>cors</code> , <code>helmet</code> -like protections, <code>jsonwebtoken</code>
Email	Nodemailer (Gmail SMTP)
Containerization	Docker (multi-stage build, non-root user) + Docker Compose
CI/CD	Jenkins (declarative pipeline)
Dev Tools	Nodemon (auto-reload)

 Prerequisites

- [Node.js](#) v18 or higher

- [MongoDB](#) (local instance or MongoDB Atlas)
- [Google Gemini API Key](#) — **required**
- [OpenAI API Key](#) — *optional, enhances evaluation accuracy*
- [Cohere API Key](#) — *optional, enables multi-AI synthesis*
- Gmail account with [App Password](#) — *for email features*

Getting Started

1. Clone the repository

```
git clone https://github.com/ketanayatti/ai-mock-interview.git
cd ai-mock-interview
```

2. Install dependencies

```
npm install
```

3. Configure environment variables

```
cp .env-example .env
```

Edit `.env` with your actual credentials:

Variable	Required	Description
PORT	No	Server port (default: 3000)
NODE_ENV	No	development or production
JWT_SECRET	Yes	Secret key for JWT token signing
SESSION_SECRET	Yes	Secret key for session cookie encryption
MONGO_URI	Yes	MongoDB connection string (local or Atlas)
EMAIL_USER	No	Gmail address for sending emails
EMAIL_PASS	No	Gmail App Password
GEMINI_API_KEY	Yes	Google Gemini API key (primary AI)
API_KEY	No	Google Gemini API key (alias)
OPENAI_API_KEY	No	OpenAI API key (enhances evaluation)
COHERE_API_KEY	No	Cohere API key (enables synthesis)

4. Start the server

Development (with auto-reload via Nodemon):

```
npm run dev
```

Production:

```
npm start
```

The app will be available at <http://localhost:3000>.

Docker Deployment

The application uses a multi-stage Docker build with a non-root user for security.

Development (with local MongoDB)

```
docker compose up -d --build
```

This starts:

- **App container** on port **3000** (configurable via **PORT** env var)
- **MongoDB 7** container on port **27017** with health checks

Production

```
docker compose -f docker-compose.prod.yml up -d --build
```

Production differences:

- App exposed on port **4000** (maps to internal **3000**)
- MongoDB port is not externally exposed
- Separate named volumes for data persistence (**mongo_data_prod**, **resume_uploads_prod**)

Stop containers

```
docker compose down  
# or for production:  
docker compose -f docker-compose.prod.yml down
```

CI/CD Pipeline (Jenkins)

The project includes a `Jenkinsfile` for automated deployment:

Branch	Action	Compose File
develop	Deploy to Staging	<code>docker-compose.yml</code>
main	Deploy to Production	<code>docker-compose.prod.yml</code>

Jenkins Credentials Required: `JWT_SECRET`, `SESSION_SECRET`, `MONGO_URI`, `GMAIL_USER`, `GMAIL_PASS`, `GEMINI_API_KEY`, `API_KEY`, `OPENAI_API_KEY`, `COHERE_API_KEY`

API Routes

Public Routes

Method	Endpoint	Description
GET	<code>/</code>	Landing page (redirects to dashboard if session exists)
GET	<code>/welcome</code>	Welcome / continue session page
POST	<code>/api/start-new</code>	Create a new session (AJAX)
POST	<code>/api/continue-session</code>	Continue an existing session (AJAX)
POST	<code>/start-new</code>	Create a new session (form-based)
POST	<code>/continue-session</code>	Continue an existing session (form-based)

Protected Routes (require active session)

Method	Endpoint	Description
GET	<code>/dashboard</code>	User dashboard showing all interview spaces
GET	<code>/profile</code>	User profile page
POST	<code>/update-profile</code>	Update user name
GET	<code>/end-session</code>	End session and clear cookies
GET	<code>/performance</code>	Performance analytics dashboard

Space Routes (protected)

Method	Endpoint	Description
POST	<code>/spaces/create</code>	Create a new interview space (with resume upload)
GET	<code>/space/:id</code>	View space details and round summaries
GET	<code>/space/resume/download/:id</code>	Download uploaded resume

Interview Routes (protected, rate-limited)

Method	Endpoint	Description
GET	/space/:spaceId/round/:roundName/start	Load interview screen UI
GET	/generate-questions/:spaceId/:roundName	Generate the first question (AI)
POST	/next-question/:spaceId/:roundName	Get the next adaptive question (AI)
POST	/finish-round/:spaceId/:roundName	Finish round and trigger 3-AI evaluation
GET	/api/questions-answers/:roundId	Retrieve Q&A history for a round

 Data Models

Session

Field	Type	Description
uniqueId	String (unique)	8-character hex session identifier
name	String	User's display name
spaces	ObjectId[]	References to interview spaces
lastActive	Date	Last activity timestamp
createdAt / updatedAt	Date	Auto-generated timestamps

Space

Field	Type	Description
studentId	String	Session's uniqueId
companyName	String	Target company name
jobPosition	String	Target job position
jobDescription	String	Job description text
experienceLevel	Enum	fresher / intermediate / experienced
interviewRounds	Array	List of rounds (HR, Technical, Final, etc.)
resumePath	String	Filename of uploaded resume
resumeText	String	Extracted text from the resume
purifiedSummary	String	AI-generated resume summary
createdAt / updatedAt	Date	Auto-generated timestamps

Interview Round (embedded in Space)

Field	Type	Description
name	String	Round name (e.g., HR, Technical)
status	String	not completed / in_progress / completed
summary	String	AI-generated evaluation summary

QuestionAnswer

Field	Type	Description
spaceId	ObjectId	Reference to the parent Space
roundName	String	Name of the interview round
question	String	The interview question
answer	String	The candidate's answer
isFollowUp	Boolean	Whether this is a follow-up question

Student (legacy model)

Field	Type	Description
name	String	Student's name
email	String (unique)	Email address
password	String (hashed)	Bcrypt-hashed password
jobPositions	String[]	Targeted job positions
profilePhoto	String	Profile photo path
geminiApiKey	String	Personal Gemini API key

 Security Features

- **Rate Limiting** — Three tiers of rate limiting:
 - General API: 100 requests / 15 minutes
 - Space creation: 5 spaces / hour (per session)
 - Interview actions: 50 requests / hour (per session)
- **Session Security** — HTTP-only, SameSite cookies with secure flag in production
- **File Upload Validation** — MIME type whitelist (PDF, DOCX only), 10MB size limit
- **Path Traversal Protection** — Filename sanitization and path resolution checks on resume downloads
- **Content Sanitization** — All rendered content sanitized with DOMPurify to prevent XSS
- **CORS** — Cross-Origin Resource Sharing enabled
- **Non-root Docker** — Production container runs as non-root user (`appuser`)

 Project Structure

```

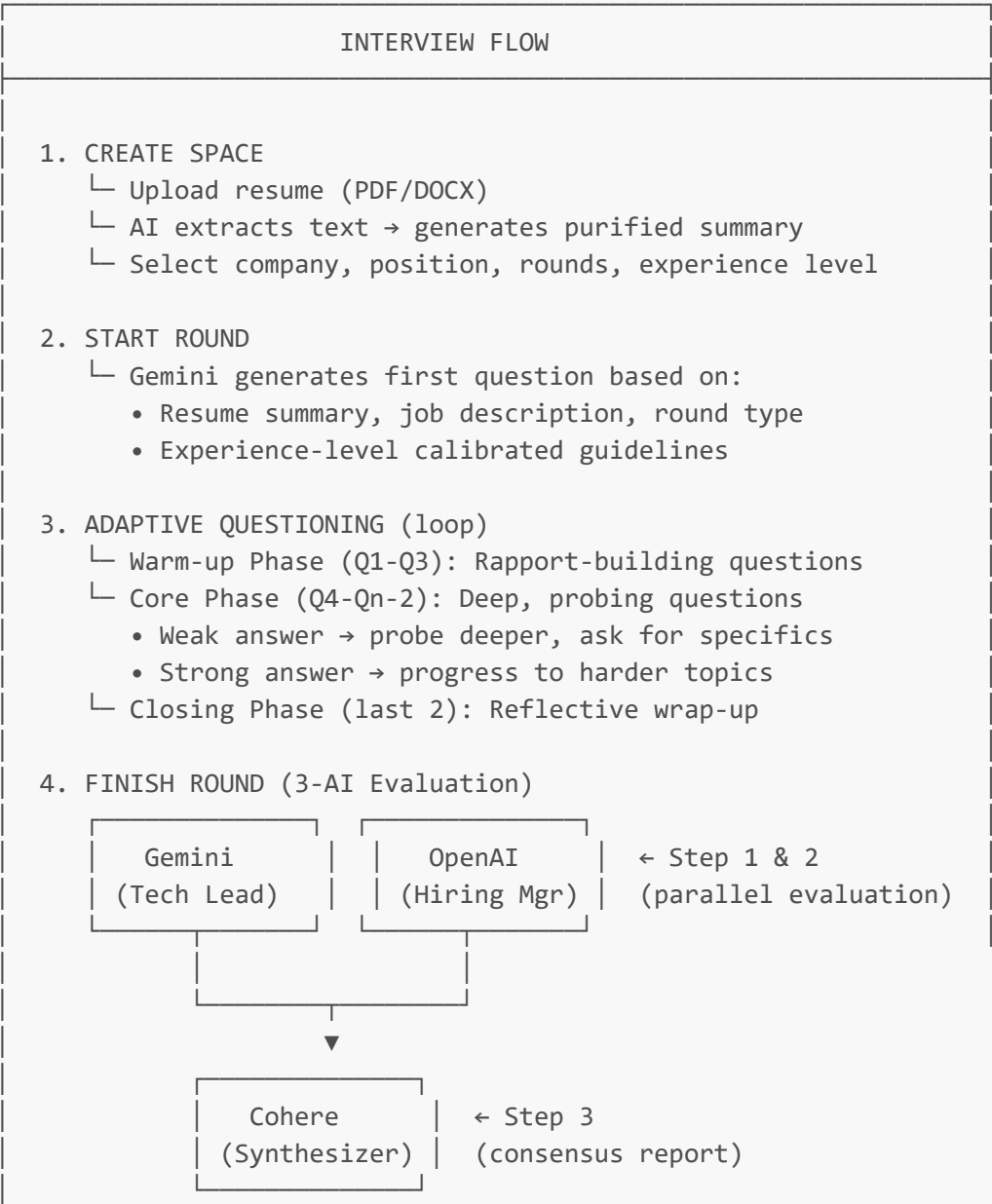
ai-mock-interview/
├── server.js                # Entry point – loads env, connects DB,
starts server
├── main.js                 # Standalone Gemini AI test script
├── package.json            # Dependencies and scripts
├── Dockerfile              # Multi-stage Docker build (Node 20 Alpine)
├── docker-compose.yml      # Dev Docker config (app + MongoDB 7)
├── docker-compose.prod.yml # Prod Docker config (port 4000, isolated
MongoDB)
├── Jenkinsfile             # CI/CD pipeline (develop → staging, main →
production)
├── .env-example            # Environment variable template
├── .gitignore              # Git ignore rules
├── .dockerignore           # Docker ignore rules
├── public/
│   ├── css/
│   │   └── style.css      # Global stylesheet
│   └── Resumes/          # Uploaded resume files (gitignored)
└── src/
    ├── app.js             # Express app configuration, middleware, rate
limiters
    ├── routes.js          # All route definitions with Multer upload
config
    │   ├── config/
    │   │   ├── aiServices.js # Centralized AI service initialization
    │   │   └── dbConfig.js   # MongoDB connection with Mongoose
    │   └── email.js         # Nodemailer Gmail transporter
    ├── controllers/
    │   ├── homeController.js # Static page rendering (home, about,
contact)
    │   ├── sessionController.js # Session CRUD (create, find, continue, end,
profile)
    │   └── spaceController.js  # Interview space management, resume parsing,
AI summarization
    │   └── interviewController.js # AI question generation, adaptive follow-
ups, 3-AI evaluation
    │   ├── models/
    │   │   ├── sessionModel.js # User session schema (uniqueId, name,
spaces)
    │   │   └── spaceModel.js   # Interview space schema with embedded round
sub-documents
    │   └── questionAnswerModel.js # Q&A tracking schema (questions, answers,
follow-ups)
    │       └── studentModel.js # Student schema with bcrypt auth (legacy)
    └── services/

```



```
├── geminiService.js      # Legacy Gemini service wrapper
├── views/
│   ├── home.ejs          # Landing page
│   ├── welcome.ejs       # Welcome / session continue page
│   ├── 404.ejs           # Not found error page
│   ├── error.ejs         # Generic error page
│   └── student/
│       ├── dashboard.ejs  # User dashboard with space cards
│       ├── interview-screen.ejs # Live interview UI
│       ├── space-details.ejs # Space detail view with round summaries
│       ├── performance.ejs # Performance analytics page
│       ├── profile.ejs    # User profile page
│       └── session-created.ejs # Session confirmation with unique ID
```

How the AI Interview Works



▼

Final Report

- Score (X/10)
- Verdict (Strong Hire / Hire / No Hire)
- Strengths & Weaknesses (with quoted evidence)
- Question-by-Question Breakdown
- Actionable Improvement Advice

Available Scripts

Script	Command	Description
Dev	<code>npm run dev</code>	Start with Nodemon (auto-reload on file changes)
Start	<code>npm start</code>	Start in production mode (<code>node server.js</code>)
Test	<code>npm test</code>	Run tests (not yet configured)

Contributing

1. Fork the repository
2. Create your feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

License

This project is licensed under the **ISC License**.